# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## ASSESSMENT TOOL 2 – CONCEPT MAPPING

| | | | |
|---|---|---|---|
| **Course Code:** | CSA06 | **Course Title:** | Design and Analysis of Algorithms |
| **CO Assessed:** | CO1 – Precision (BL1, BL2, BL3) | **Assessment:** | Assessment Tool 2 – Concept Mapping |
| **Weightage:** | 40% of CIA | **Total Marks:** | 100 (2 Questions × 50 Marks) |
| **CO1 Statement:** | *Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)* | | |
| **Student Name:** | Kishore Gadhi | **Reg. No.:** | 192365051 |
| **Section / Batch:** | | **Date:** | |

**Instructions to Students**

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

| Question Type | Focus Area | Questions | Marks |
|---|---|---|---|
| CONCEPT MAPPING | Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method | Q1 – Q2 | Q1 –50<br>Q2 –50 |
| | **GRAND TOTAL:** | **100 Marks** | |

## SECTION A – CONCEPT MAPPING

**Explain how recursion depth affects stack memory usage. Extend the map by including input size dependency. Justify why deep recursion increases memory usage. Interpret how this mapping helps optimize recursive solutions.**

**Code of Conduct**

*I KISHORE GADHI (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

## RUBRICS FOR CALCULATION

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Problem Context | 20% | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem |
| Application of Concepts | 30% | Accurately applies appropriate concepts to solve complex problems | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy | Fails to apply appropriate concepts |
| Problem-Solving Approach | 20% | Logical, well-structured, and efficient approach | Mostly logical approach with minor gaps | Basic approach with some inconsistencies | Illogical or unclear approach |
| Accuracy of Solution | 20% | Completely correct solution with proper justification | Mostly correct with minor errors | Partially correct solution | Incorrect or incomplete solution |
| Clarity | 10% | Clear, well-organized, and properly explained solution | Understandable with minor clarity issues | Limited clarity and explanation | Poorly presented and difficult to understand |

**Marks Evaluation:**

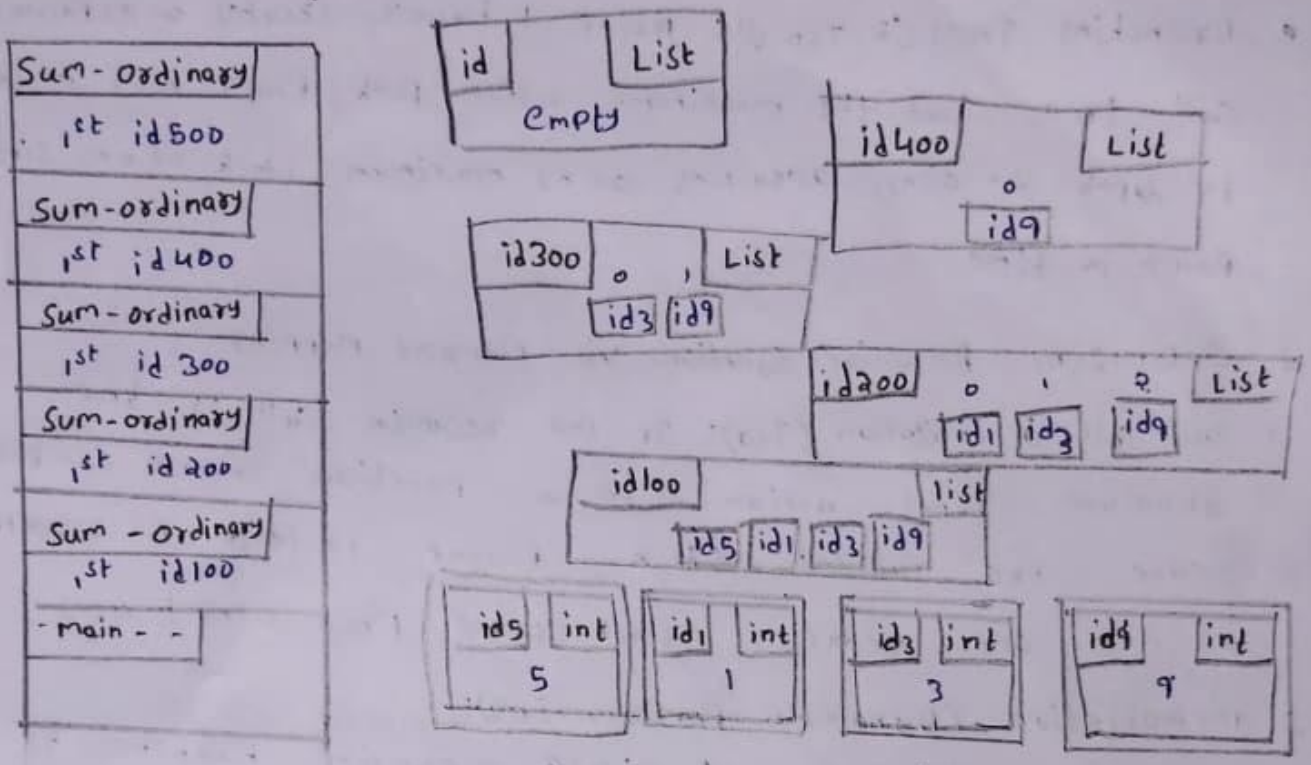
| Question No. | Understanding of Problem Context (20%) | Application of Concepts (30%) | Problem-Solving Approach (20%) | Accuracy of Solution (20%) | Clarity (10%) | Total Marks |
|---|---|---|---|---|---|---|
| **Q1** (50 Marks) | | | | | | |
| **Q2** (50 Marks) | | | | | | |
| | | | | | **Overall Total (100)** | |

| Faculty In-charge | Course Coordinator | Head of Department |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

G. Kishore
19236505 L

# How Recursion Depth Affects, stack memory

Every time a recursive function calls itself, the system allocates a stack frame on the execution stack.




Call stack growth with recursive depth,

What an Activation Frame Contains.

- Return address : Instruction on where to resume once the child call return.
- Local Variable & arguments : Parameters Passed into the Frame
- Saved registered state : Internal register references needed to restore Context.

The Input Dependency.

The maximum depth of the call tree (d) directly dictates total auxiliary space Complexity S(n):

Total Stack Memory = d x (size of Single Frame)

- Linear Decrement ($n \to n-1$): standard factorial or linear search Creates a call stack of depth $d = O(n)$, Consuming $O(n)$ stack space.

- Logarithmic Division ($n \to n/2$): Binary Search or divide-and-Conquer splits input in half, capping recursion depth at $d = O(\log n)$, Consuming $O(\log n)$ Space.
- Branching Trees ($2.T(n-1)$): Naive fibonacci Creates a exponential Call tree, but the maximum active Path from root to leaf is depth $d = O(n)$, requiring $O(n)$ maximum stack at any single Point in time

2. Optimizing Recursive Solutions via Memory Mapping

1. Tail Call optimization (TCO): If the recursive call is the absolute final action in a function, smart compilers reuse the current stack frame instead of allocating a new one; reducing stack space From $O(n)$ to $O(1)$

2. memoization (Dynamic Programming):
Eliminates duplicate recursive branches by storing prior subproblem outputs, Flattening subproblem dependency trees.

3. Iterative Conversion: Replace stack frame allocations entirely with Explicit data structures or simple loops on the heap.

3. Determining Algorithm Efficiency

To mathematically evaluate recursive algorithms, we analyze time complexity using Asymtotic Notations ($O, \Omega, \Theta$) and solve recurrence relation

A. The Substitution Method (Mathematical Induction)
The subsitution method involves guessing the bound and using mathematical induction to Prove it correct.

Example: Solve $T(n) = T(n-1) + n$

1. Expansion: $T(n) = (T(n-2) + n-1) + n$

$= T(n-2) + (n-1) + n$.

Extrapolating down to base case $T(1) = 1$:

$$T(n) = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = O(n^2)$$

2. Inductive Proof:

Guess that $T(n) \leq cn^2$ for some constant $c > 0$

- Inductive Hypothesis: Assume $T(k) \leq ck^2$ for all $k < n$.
- Inductive step:

$$T(n) = T(n-1) + n \leq c(n-1)^2 + n$$

$$T(n) \leq c(n^2 - 2n + 1) + n = cn^2 - 2cn + c + n$$

$$T(n) \leq cn^2 - n(2c-1) + c$$

for $T(n) \leq cn^2$, we require $-n(2c-1) + c \leq 0$, which holds true for $c \geq 1$ and sufficiently large $n$. Thus $T(n) = O(n^2)$

3. The Master theorem

The Master Theorem provides a quick asymtotic solution for divide and conquer recurrences of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n) \quad (a \geq 1, b > 1)$$

We compare $f(n)$ with $n^{\log_b a}$:

| Case | Condition | Solution |
|---|---|---|
| 1 | $f(n) = O(n^{\log_b a - \epsilon})$ for $\epsilon > 0$ | $T(n) = \Theta(n^{\log_b a})$ |
| 2 | $f(n) = \Theta(n^{\log_b a} \log^k n)$ for $k \geq 0$ | $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$ |
| 3 | $f(n) = \Omega(n^{\log_b a + \epsilon})$ AND $af(n/b) \leq cf(n)$ for $c < 1$ | $T(n) = \Theta(f(n))$ |

Applied example: Merge sort

$$T(n) = 2T\left(\frac{n}{2}\right) + \theta(n)$$

- $a = 2 \quad b = 2, \quad F(n) = n$
- Calculate $n^{\log_b a} = n^{\log_2 2} = n^1 = n$
- since $F(n) = \theta(n^{\log_b a})$, this falls under case 2 $(k = 0)$

Result: $T(n) = \theta(n \log n)$